

Programming Language Design

Module 5: Lisp, Lexing and Functional Operators

Slides © Grant Williams, [CC BY-SA 4.0](#).

This is an open educational resource.

Feel free to submit fixes, improvements, and new material [here](#).

Let's make a programming language

This class isn't "learning a weird language", it's "programming language design". Let's start designing!

We're going to make a simple version of the Lisp programming language.

But how do we do that? What does it even mean to design a programming language?

Designing a programming language

A programming language is a formal language. It has rules that describe its grammar.

We could specify a formal grammar (and we will), but for now, let's get a "feel" for this language.

The language we'll be working with is called [Lisp](#), which stands for "List Processor", but sometimes detractors say it stands for "lots of irritating, superfluous parentheses".

Lisp is an extremely influential language, and is still widely used today, mainly as a scripting language (used in [Emacs](#), [AutoCAD](#), and [Audacity](#)), but also as a full-on programming language ([Common Lisp](#), [Scheme](#), [Racket](#), [Clojure](#))

Lisp dialects

Lisp was released as a single language, but soon, "dialects" emerged.

In human languages, "dialect" is a term for a language that is considered by some speakers to be a kind of "version" of another language. Often a variation that is only common in a smaller region or population relative to the language being compared to.

Of course, which gets to be the "real" language and which is the "dialect" is subjective and a matter of perspective. For this reason, it can be controversial to refer to a language as a "dialect", which is why I don't have any examples listed here.

This controversy does not extend to computer languages, and computer scientists who modify or extend another language will happily refer to their creation as a "dialect".

Lisp dialects (2)

Lisp has lots of dialects. In fact, I have never seen someone program in actual *Lisp*. People are always using a dialect when they write new Lisp whenever I see it written.

We will develop a simple Lisp dialect called *Slisp*, which just stands for "Simple Lisp".

The name is a bit of a lie. It's really more like the language *Scheme*, which is a functional dialect of lisp. However, "SScheme" doesn't roll off the tongue.

Lisp traits: type system

As programming language designers, let's learn about some important Lisp attributes:

Lisp dialects are *usually* dynamically typed. Exceptions exist, but if there are typing features, they are usually *gradual*, meaning that they can optionally be added later.

There are a few simple value types built in, called *atoms*. Integers and "symbols" (meaning, actual words) are both atoms. Floats could be atoms, too.

The basic data structure is the list, as you'd expect from the name of the language.

Lisp lists

Lists are implemented as singly-linked lists. They are written like this:

```
(1 2 3 4 5)
```

That's it. Just space-separated values inside of parentheses.

Lists can be nested inside of other lists:

```
((1 2 3) 4 5)
```

.

An expression written in this syntax is called an [S-expression](#).

Lisp Programs

In every Lisp dialect I am aware of, a Lisp program is simply one or more s-expressions.

These s-expressions are evaluated by default by treating the first element as a function, and the other elements as arguments.

So we would not write `(1 2 3 4 5)` by itself in any Lisp dialect, because it would try to call `1` as a function on values `2 3 4 5`, which would be an error.

If you want a list that isn't executed like that, there's a way to get it, but we'll get there later...

Hello world

For now, as is tradition, let's write a simple hello world program in slisp:

```
(print "hello, world")
```

Two things are important here:

1. This is a complete program. It consists of one s-expression.
2. The s-expression's first and only argument is the string "hello, world".

How can we make it more interesting? Let's do some math?

Hello world: math edition

```
(print "hello, world. A neat number is:" (+ 1 2 3 1))
```

Here, `(+ 1 2 3 1)` is an expression that is interpreted as an argument to `print`. We would expect this to print:

```
hello, world. A neat number is: 7
```

In this case `+` is a function call. The rest of the numbers after it are interpreted as arguments, and the result is that they are all summed together.

That's enough Lisp for now

That's all we'll say about lisp for now, but we'll spend a lot more time refining our implementation.

By the end of the course, we'll have implemented an interpreter for Slisp.

That means you can feed Slisp programs to it and it will execute them.

This interpreter will be written in Haskell. You will be using a language you don't know to write an interpreter for a language you don't know.

In so doing you will become enlightened. Just kidding, but you will know a couple of obscure, cool languages, and have the tools you need to explore language design further.

Questions?

Formal language processing

Using a programming language in practice typically involves the following phases:

1. Lexical analysis (aka lexing, tokenization)
2. Parsing
3. Compilation (optional)
4. Execution (aka interpretation unless we're using hardware)

Let's talk about the first step.

Lexing

How can we interpret `(print "hello, world. A neat number is:" (+ 1 2 3 1))` ?

Our first step is called *lexing*.

Lexing means breaking a string up into *lexemes*, also called *tokens*, which are basically like "words". They are kind of the building blocks of programs.

Tokenization is another word for lexing. The term *token* has entered popular knowledge because tokenization is something LLMs do, and they typically charge per token.

Lexing in formal languages

With LLMs in English, a token is one or more letters, like "-tion". Words are built out of tokens.

In formal languages, we rarely break down "words" into sub-words. Most programming languages are highly analytic, meaning that they can be broken down into tiny "words" that mean very exact things, rather than grouping lots of related thoughts into one word.

Most "words" from a programming language perspective are either operators, functions, or values.

Example lexing

For example, the program `(print "hello, world. A neat number is:" (+ 1 2 3 1))` can be lexed like this:

```
[ (, print, "hello, world. A neat number is:", (, +, 1, 2, 3, 1, ), ) ]
```

Here, I broke up each token with a comma.

Please notice the following:

- Grouping operators like `(` and `)` are individual tokens.
- The whole string is one token. The programming language doesn't know how to interpret the words inside the string, so the whole thing is just one value.
- Integer literals are tokens
- The `+` operator is a token. Actually any operator.

How do we do that?

Now is a good time to look at project 2.

Your job is to implement a lexer.

But how do you do that?

Our job is to implement a function that takes a string (representing the program code), and returns a list of tokens.

```
tokenize :: String -> [Token]
```

Identifying the kinds of tokens

Slisp has very few kinds of tokens. Here's a list:

- Symbols are names of variables or functions. For example `print` is a symbol.
- String literals are a kind of token. For example, `"hello, world"` is a token.
- Integer literals, too. For example `123` and `-123` are both tokens.
- Parentheses are tokens. `(` and `)`.
- Comments are not tokens, but the lexer usually strips them out, so we need to handle them.

What about other things?

Is that it? Those are the only things we care about?

What about functions? What about loops?

Those things are not tokens! The word `while` might be a token in C, but remember that tokens are just "interesting words". An entire function or loop will require many tokens.

Right now, we're just trying to lex through a slisp program and return a list of tokens. That's all a tokenizer is:

```
tokenize :: String -> [Token]
```

Our next slides will drop hints about how to implement this function.

Questions?

Lexing lisp: comments

Lisp is a very easy language to lex.

First of all, there are comments. We have already seen that line comments are written with `;`.*

The lexer usually strips comments out. [Some languages](#) consider comments to be tokens, but this is rare.

Comments aren't super interesting, but it's worth considering how we stripped them out. [Can anyone share?]

* I did not make you deal with block comments because I am merciful (the interaction between `;` and block comments requires state-machine thinking which is a little tough when you're warming up to Haskell)

Lexing lisp: parentheses

Comments we just stripped out, but lexing other tokens is more complicated.

First, we have a simple rule: (and) are always tokens. So if we see one of those characters, we add it to the list of tokens...

...unless it's in a string. Okay, still not too bad.

So we need a special rule for parsing strings. But other than that, parentheses are always a single token each.

Lexing in general: literals

One vocabulary term we need right now: *literals*.

`123` is an integer literal. In C, in Haskell, and even in Lisp.

`"Hello, world"` is a string literal in those same 3 languages.

However, in this C code:

```
char* s = "hi";
```

`s` is a string, sure, but it is not a literal. `"hi"` is a literal.

A literal is an actual data value inserted into a program's source code, it is not a variable. [Is `Pi` a literal in Haskell? What about `3.1415926` ?]

Lexing lisp: string literals

A string literal starts with a quote: `"`.

Inside the string, every character belongs to the string.

If there is a `\`, we treat it as an escape sequence. The only escape sequence I will require handling is `\"` for quotes in a quoted string literal.

Otherwise, everything is in the string until we find another pair of quotes.

Lexing lisp: integer literals

First, an integer could start with a minus sign: `-123` is an integer.

But if we see a minus sign by itself, it's not an integer.

In `(- 3 2)`, `-` is a symbol that refers to subtraction.

Lexing lisp: a combined rule

So we have a slightly more complex rule we have to follow:

- If we see, whitespace outside of any lexeme, just ignore it.
- If we see a parenthesis, that's a token.
- If we see a minus sign, it could be an operator like `-` by itself, but it could also be the start of a negative integer.
- If we see a numeral, we know it should be an integer (assuming the same rules as C or Haskell, where we can't start a variable name with a numeral)
- Otherwise, it's a symbol (just a name for something, like an operator or a function)

Lexing lisp: kinds of token

So what is a `Token` in code?

[How should we represent that type?]

Lexing lisp: the datatype

```
data Token =  
    Lp          -- left paren  
  | Rp          -- right paren  
  | Sym String  -- symbol  
  | IntLit Integer -- integer literal  
  | StrLit String -- string literal  
  deriving Show
```

A sum type is perfect here. We list every possible kind of token.

Some constructors take additional information. For example, if we have a symbol or a string literal, we also store a string that contains the actual text of the symbol or literal. If we have an integer literal, we store the integer alongside it.

But if we have a parenthesis, there is nothing to store. So the constructor has no fields.

Token types

I'd like you to make sure you understand the previous slide, and please ask questions if you have any.

One random thing I want to mention: in real programming languages, the token type is a bit more complicated.

In addition to the token type and data, the token data structure usually stores at least the line number and the column so that error messages can list that information.

Also, Lisp has remarkably little token variety. Normal languages have tokens for every keyword. But you'll see that we don't really need that.

Questions?

Actually lexing

Now that you understand the datatypes you'll need, let's think about the function that does lexing. We call it *tokenize* (although it could also be called *lex*).

Recall the type:

```
tokenize :: String -> [Token]
```

It takes a string representing the program and it returns a list of tokens.

The same token values we enumerated in the data declaration earlier.

For example:

```
tokenize "(print \"hello\" (+ 2 2))" ==  
  [Lp, StrLit "hello", Lp, Sym "+", IntLit 2, IntLit 2, Rp, Rp]
```

What goes in that function?

That's the question you'll need to answer to complete the next project!

We need some basic rules that can not only split the code into tokens, but also determine the correct type of function.

To do this, I have some suggestions...

A good basic design

Instead of considering how to split the entire code into tokens all in one step, lets apply some functional programming software design, and imagine some simpler functions that could be useful.

I suggest this one:

```
nextToken :: String -> (String, Maybe Token)
```

This function will run on a string. It will determine the next token if it exists, and return two things:

- The remaining string
- `Just` that token.

If the token doesn't exist, it returns `("", Nothing)`

What goes in `nextToken`?

I recommend some pattern matching:

- If the first character of the string is whitespace, skip it.
- If the first character is a parenthesis, do this:

```
nextToken '(' : rest) = (rest, Lp)
```

Here, `Lp` is the constructor for the left parenthesis token. How do you handle `)`?

- What if the character is a digit? Then call a `nextInteger` function. You can write this yourself, but I recommend using the `break` function we will learn.
- What if the character is a `-`? Then it could be an integer or a symbol. You'll need to check the next character after the `-` if it exists.
- If the character is a `"`, it's a string. You'll need to figure out how to lex the string.
- If it isn't anything else, it's a symbol.

Questions?

Functional operators

The assignment can be done purely with recursion and with functions we've seen so far.

However, if you've been following my advice about going to competitive coding sites and grinding some Haskell practice, you've probably seen ridiculously tiny programs that do an unreasonable amount of work.

This is the magic of functional programming, and what's especially cool, is that once you get used to it you will find many of these programs to be easy to read.

These programs use *higher order functions*. These are functions that take other functions as arguments. Some of these are so standard, we call them *functional operators*.

Examples: upper casing

Suppose I want to convert a string to uppercase. I could do this:

```
import Data.Char -- for toUpper
upperCase "" = ""
upperCase (h : t) = toUpper h : upperCase t
```

First, make sure you understand how it works...

It's a little wordy though.

Easier upper casing

We can do this instead:

```
import Data.Char
uppercase str = map toUpper str
```

Or, even nicer, point free (let's assume `Data.Char` is imported from now on):

```
uppercase = map toUpper
```

`map` is one of those functional operators. It's so influential, even imperative languages have it now. For example: Python has it. Let's learn how it works.

How `map` works

`map` is a higher order function.

It takes a function and a list, and it runs the function on every element of the list.

Here is its type:

```
map :: (a -> b) -> [a] -> [b]
```

That is, it takes a function that takes us from type `a` to type `b`. Then it takes a list of `a` s, and it will run the function on all of them to produce a list of `b` s.

Some map examples

Suppose I want to add 1 to every value in a list. There's the awkward way:

```
plus1 :: [Int] -> [Int]
plus1 [] = []
plus1 (h : t) = (h + 1) : plus 1 t
```

And there's the cool way:

```
plus1 :: [Int] -> [Int]
plus1 = map (+1)
```

Both functions do the same thing. But the second will get upvotes on CodeWars.

Knowledge check 1

1. Write a function with `map` that takes a list of integers and returns a new list in which each integer is doubled.
2. Write a function with `map` that takes a list of doubles and squares them all.
3. Write a function and its type: the function uses `map` to take a string and invert its case. So lower case letters become upper case, upper case letters become lowercase, and any symbol that is not a Latin letter is passed through without change. You will want a helper function that does the inverting. Use `isLower`, `isUpper`, `toLower`, and `toUpper`.
4. (tricky) define a list, not a function, which is the list of squared natural numbers. (i.e., 0, 1, 4, 9, 16, etc.)

KC 1 answers:

1. `doubleEach = map (*2)`

2. `squareEach = map (**2)`

3.

```
invertStr :: String -> String
invertStr = map invert
  where
    invert c
      | isLower c = toUpper c
      | isUpper c = toLower c
      | otherwise = c
```

4. `natSquares = map (^2) [0..]`

How does map work?

So map runs a function on a list and returns the new list.

But how does it work? Is it built-in to the language?

No, actually. It's a function written in Haskell.

How do we write it? How do we write a function that takes another function and runs it on every element of a list?

This is something to bear in mind about Haskell: almost all of its "features" are just functions, and we can easily write them and learn how to extend the language.

So, let's extend it. First, [remind me what the type of `map` is.]

Map's type and base case

```
map :: (a -> b) -> [a] -> [b]
```

It takes a function and a list and returns a list.

We can implement it using recursion. The base case makes sense:

```
map' _ [] = []
```

That is, if the list is empty, ignore the function and return the empty list.

[What's the recursive case?]

Map's recursive case

```
map' f (x : xs) = f x : map f xs
```

That's it. We run `f` on the first element of the list.

Then we append the result with the result of mapping the rest of the list.

Simple, recursive, and very useful. We can extend the language by adding functions.

Let's add some more. But first...

Questions?

Take

Let's consider that last knowledge check before. The one where we wanted the list of all the squares of natural numbers:

```
natSquares :: [Integer]
natSquares = map (^2) [0..]
```

The reason this works is that Haskell is lazily evaluated. In an eager language without iterators, `map (^2) [0..]` would run forever.

But that also means we can't easily work with this list. If we print it, the program will run forever:

```
print natSquares -- <-- this loops forever
```

Take (2)

Enter `take`. This is a simple functional operator that takes a certain number of elements from a list.

```
print $ take 10 natSquares
```

`take 10 natSquares` results in `[0,1,4,9,16,25,36,49,64,81]`, which is what gets printed.

So how does this work? How is `take` able to "stop" after a certain number of elements?

First, let's consider its type. What is the type of `take`?

Take (3)

```
take :: Int -> [a] -> [a]
```

Give me a number and a list of something and I will give you a list of that many somethings.

What if there aren't that many things? Then return as many as there are.

Okay, now you do the rest. Please implement the *three* cases of `take`. Yes. it has two base-cases.

Take (4)

```
take 0 _ = []  
take _ [] = []
```

These are the base cases. If we take 0 elements, we get an empty list. If we take any number from an empty list, we get an empty list.

Otherwise:

```
take n (h : t) = h : (take (n - 1) t)
```

Add the head to the result, and take one fewer from the tail.

(I've ignored the negative case. In our case, it will just keep taking. How could I handle

`take -1` if I wanted to?)

Knowledge check 2

1. Implement `drop` which is the opposite of `take`. Instead of taking three items, `drop 3` skips over the next three items and returns the rest of the list. Include its type.
2. What would `drop 3 $ take 10 $ map (*2) [1..]` return?
3. What would `take 10 $ drop 3 $ map (*2) [1..]` return?
4. Why do you think `take` and `skip` use `Int` instead of `Integer`?

KC2 answers

1.

```
drop' :: Int -> [a] -> [a]
drop' 0 l = l -- only one base case this time
drop' n (h : t) = drop' (n - 1) t
```

2. [8, 10, 12, 14, 16, 18, 20]

3. [8,10,12,14,16,18,20,22,24,26]

4. `Integer` would work fine, but it would add a little overhead every time we subtract (checking to see if the int is big or not before indexing into it). Maybe the *prelude* (i.e., functions available by default) designers didn't think it was worth the overhead. I actually can't find a source here. Alternatively, maybe `Integer` wasn't around when `take` was added, and they didn't want to break code.

Questions?

Filter

Sometimes we want all of the members of a collection that satisfy a requirement.

For example: take only the lowercase letters from a string:

```
filter isLower "Hello, Friends!" == "elloriends"
```

Or, maybe take only the non-uppercase letters. Not the same thing!

```
filter (not . isUpper) "Hello, Friends!" == "ello, riends!"
```

It's clear that filter is "filtering", but how does it work? What is its type?

Filter (2)

Filter's type is:

```
filter :: (a -> Bool) -> [a] -> [a]
```

That is, it takes a *predicate* that makes statements about an `a`. A function that returns `Bool` is called a *predicate*. Here, the function is telling us whether the `a` is one we want to keep or not.

Then it takes a list of `a`s. But *only* the ones that the predicate returned true for.

So how is it implemented?

Filter (3)

```
filter _ [] = []  
filter f (x : xs) =  
    if f x then x : filter f xs  
    else filter f xs
```

The base case is when the list is empty. The result is an empty list.

When there's at least one value in the list, we run the function on it. If it's true, we keep that value by appending it to the result. If it's false, we just ignore it and filter the rest of the list.

Knowledge check 3

1. Define a function that only takes even elements from a list of integers.
2. Define a function that only takes floats from a list if the float in question is greater than `3.0`.
3. Define a function that is the opposite of filter. That is, it only takes a value if the predicate *fails*.
4. Define a function that returns the count of uppercase Latin letters in a string.

Knowledge check 3 answers

1. `filter ((== 0) . (`rem` 2))` , or, less fancily: `filter (\x -> x `rem` 2 == 0)`
2. `filter (> 3.0)`
3. `filterNot f = filter (not . f)` . You could make it recursive, too, but isn't this cleaner? Just invert `f` by composing it with `not` .
4. `countUpper = length . filter isUpper`

Here, we first filter out only the letters that are upper case. Then, we return the length of that string.

Questions?

Fold and friends

It's nice that we can take a list and either modify each element in isolation, or take only certain elements.

But sometimes, we want to collapse a list down into a single value.

When? Consider these three functions...

Sum, product, concat

1. I want to add a list of integers together. How do I do it?
2. I want to multiply a list of integers together. How do I do it?
3. I want to concatenate a list of strings together. How do I do it?

Sum

Here's a recursive solution:

```
sum' :: [Integer] -> Integer
sum' [] = 0
sum' (x : xs) = x + sum' xs
```

There's a prime (') after the name because this function is already built-in.

Does this solution make sense?

[How should product work?]

Product

```
product' :: [Integer] -> Integer
product' [] = 1
product' (x : xs) = x * product' xs
```

Here, I made the decision to have the empty list have product 1. If I made it zero, then it wouldn't work as a base case (we would always be multiplying by zero).

We could still have that case, but then we'd need two base cases. One for the empty list (0) and one for a singleton list (the value).

How do we do string concatenation?

Concat

```
concat' :: [String] -> String  
concat' [] = ""  
concat' (x : xs) = x ++ concat' xs
```

Concatenating an empty list gives us an empty string.

If the list has a string in it, we paste it to the beginning of the result string we get from concatenating the rest.

So `concat' ["hey", "there", "hi", "there"] == "heytherehithere"`

Questions?

Did anyone notice a pattern between those three, btw?

The pattern

First, let's look at the base cases:

```
sum' [] = 0  
product' [] = 1  
concat' [] = ""
```

The only difference is the "base" value.

What about the recursive cases?

The pattern (2)

```
sum' (x : xs) = x + sum' xs  
product' (x : xs) = x * product' xs  
concat' (x : xs) = x ++ concat' xs
```

Here, the only difference is the function we use to simplify the list.

For sums, we're adding everything together with `+`.

For products, we're using `*`.

And for concat, we're using `++`.

The pattern (3)

Think of it like this: we're just changing the operator that is getting inserted into the list.

```
sum'      [1, 2, 3]    == 1 + 2 + 3
product'  [1, 2, 3]    == 1 * 2 * 3
concat'   ["1", "2", "3"] == "1" ++ "2" ++ "3"
```

This is an extremely common pattern. *Reducing* a list by inserting a binary operator into it. In some languages (like Ruby and Python), it is literally called "reduce".

In Haskell, this operation is called "fold".

Let's define a fold operator that, given a base-case value and a binary operation, it applies that operation to the list over and over until it is converted into a single value.

First, what type should it have?

Our fold

This type reflects the fold operation as it was for `sum'`, `product'`, and `concat'`:

```
fold :: a -> (a -> a -> a) -> [a] -> a
```

It takes a base value (a value for the empty list), a *binary* function (in this case a function that takes two `a` s and returns an `a`), and a list of `a` s.

It returns an `a` : the result of baking down the whole list into a single value.

Note: the real fold's type is a bit more flexible than this. We'll develop it.

How do we implement this function?

Fold

```
fold base _ [] = base  
fold base f (x : xs) = f x (fold base f xs)
```

If we fold an empty list, we get the value from the base case.

If we fold a list with a head and tail, we apply the binary function to the head, and the result of folding the tail.

Using fold in our definitions

So, now we can redefine `sum'`, `product'`, and `concat'` all in terms of folding:

- `sum' = fold 0 (+)`
- `product' = fold 1 (*)`
- `concat' = fold [] (++)`

All we do is specify the base case and the binary operator, and we have enough information to reduce a list down to a single value.

Questions?

Fold is a bit more complex

All those examples were fairly simple, but fold is a bit more flexible.

First, if you've tried to call a function called "fold" in Haskell, you've probably noticed that it doesn't exist.

That's because there are two versions: `foldl` and `foldr` that assume different associativities about the binary operator.

Why would that matter?

Not every operator is associative

Suppose we want to reduce a list with the `-` operator:

```
100 - 9 - 8 - 7 == 91 - 8 - 7 == 83 - 7 = 76
```

This makes sense, but the problem is that `-` is *not* an associative operator. Let's consider what happens when we run fold on it:

```
fold 0 (-) [100, 9, 8, 7]
```

How does this simplify? Remember that in functional programming, we can replace a function call with its appropriate definition.

Associativity (2)

```
fold 0 (-) [100, 9, 8, 7] ==  
100 - (fold 0 (-) [9, 8, 7]) ==  
100 - (9 - fold (-) 0 [8, 7]) ==  
100 - (9 - (8 - (fold (-) 0 [7]))) ==  
100 - (9 - (8 - (7 - fold (-) 0 []))) ==  
100 - (9 - (8 - (7 - 0))) ==  
92
```

We get the wrong answer!

Why? Because we associate to the right: $100 - (9 - (8 - 7))$

But the real minus sign associates to the left: $((100 - 9) - 8) - 7$

And `-` is *not* an associative operator! [What does that mean?]

Associativity (3)

An associative operator `R` is one where this property is true:

$$(a \ R \ b) \ R \ c == a \ R \ (b \ R \ c)$$

That is, it doesn't matter in what order we process the operations.

The order of the operands *does* matter. We don't assume the operator is commutative.

Associativity example

`++` (string concatenation) is an operator that is associative but not commutative.

```
("Hello," ++ " World") ++ "!" == "Hello," ++ (" World" ++ "!")
```

but

```
(" World," ++ "Hello,") ++ "!" != ("Hello," ++ " World") ++ "!"
```

Because it is associative, it doesn't matter what order the `++` s get processed when we inject them into a list of strings with `fold` .

But back to the question: how do we fix folding with `-` ?

foldl

This function actually exists. `foldl` is a built-in function.

```
foldl (-) 0 [100,9,8,7] == (100 - (9 - (8 - (7 - 0)))) ==
```

Haskell puts the "starting value" second. We put it first because it matched the order of definition, but Haskell puts it second.

This function works similarly to our `fold`, but it's a bit more flexible. This is its (abridged) type:

```
(b -> a -> b) -> b -> [a] -> b
```

Why is there a `b` there? Ours only had `a`.

foldl example

Here's an example of where that's useful. Suppose I want to compute the total number of characters in a list of strings. That is, I want to sum all the lengths together.

In this case, my *starting value* is not a string, it's a length, 0. And my function takes a size for its left argument, but a string for its right:

```
foldl (\l r -> l + length r) 0 ["hey", "there", "hi", "there"] == 15
```

What's going on here? It's doing this:

```
0 + length "hey" + length "there" + length "hi" + length "there"
```

For `foldl`, the starting value is the left-most value, so it's the first value of `1`. Then, after adding `length "hey"`, that becomes the new `1` value...

foldl example (2)

```
((((0 + length "hey") + length "there") + length "hi") + length "there")
```

First, `1 == 0` and `r == "hey"`. Then the binary function adds `1` to the length of `r`, and the new value of `1 == 3`. Then we add `1 + length "there" == 3 + 5 == 8`, and that becomes the new `1`. Then we add `8 + length "hi" == 8 + 2 == 10` and that becomes the new `1`. Finally we add `10 + length "there"` to finish up with `15`.

Questions?

foldr?

What about this problem?

Define a function that computes $a \wedge b \wedge c \wedge \dots$ for a given list of exponents?

So, $\text{exp}' [2, 3, 2] == 2 \wedge 3 \wedge 2 == 512$

Wait, is that correct?

How does exponentiation associate?

foldr (2)

Exponentiation is not associative:

- $(2^3)^2 == 8^2 == 64$
- $2^{(3^2)} == 2^9 == 512$

Conventionally, we consider $^$ to be a right-associative operator in most programming languages.

So here, we *do not* want to use `foldl`, we want to use `foldr`:

```
exp' = foldr (^) 1
```

```
exp' [2,3,2] == 2^(3^(2^1)) == 512
```

foldr (3)

Notice that the "starting value" is on the right this time.

This means that the first time the binary function runs, the starting value will be `r` instead of `1`.

It also means that `foldr` has a slightly different type than `foldl`:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldl :: (b -> a -> b) -> b -> [a] -> b
```

Normally this doesn't matter, but if your "accumulator" value has a different type than your list elements, you need to know whether the previous accumulator value is in the left or right operand of the binary function.

Questions?

Knowledge check 4

1. Use filter and length to count how many letter `a` s a string has.
2. Now do the same thing with one of the `fold` s. Does it matter which one?
3. Write a reverse function. It's likely to be quadratic time unless you already know tail recursion. Include its type.
4. Now, write reverse in terms of one of the folds. Does it matter which one?

KC 4 answers

1. `countA = length . filter (=='a')`
2. `countA = foldl (\count letter -> count + if letter == 'a' then 1 else 0) 0`
(note: it looks gnarlier, but this solution only traverses the list once. It doesn't matter whether we use `foldl` or `foldr` for this one)

3.

```
rev :: [a] -> [a]
rev [] = []
rev (x : xs) = rev xs ++ [x]
```

4. `rev = foldl (\acc letter -> letter : acc) []`

It matters here. If we used `foldr`, we'd end up not reversing the list, because it would push from right to left instead of left to right.

Two more operators: span and break

I mentioned earlier that there was a helpful operator for doing lexing.

Technically it's a complimentary pair: `span` and `break`. Both are part of the prelude, so you don't have to import anything.

Both have the same type:

```
(a -> Bool) -> [a] -> ([a], [a])
```

Huh...how do we interpret that type? Anyone want to tell me what this function takes and what it returns?

Span and break (2)

`span` and `break` take a predicate (a function that returns a boolean) and a list.

They return two lists.

For `span`, it will keep running the predicate over and over on each value in the list. As long as the predicate is true, it will keep running.

As soon as the predicate is false, the function stops. It returns all the values it took when the predicate was true as the first value of the tuple, and the remaining values in the second value of the tuple.

Span and break (3)

For example, let's take numbers as long as they're less than 10:

```
span (<10) [1, 2, 5, 6, 20, 23, 25, 7] == ([1, 2, 5, 6], [20, 23, 25, 7])
```

The predicate is true for the first 4 values, so they go in the first returned list.

After that, everything else goes in the second list.

Note: this is different from `filter` !

```
filter (<10) [1,2,5,6,20,23,25,7] == [1,2,5,6,7]
```

Filter searches through *all* the values. `span` only keeps values until the predicate stops being true. As soon as the predicate is false (e.g., for the value `20`), it stops immediately and returns the rest.

Why is this useful?

Suppose I want to keep pulling values from a string as long as there are digits:
what would `span isDigit "123 456 symbol"` return?

This is a big hint for your project.

Alternatively, you can use `break`, which is the complement of `span`. It keeps pulling values as long as the predicate is *false*, and then returns as soon as its true. The first value of the result is a list of values when it was false, and the second value is the remaining values.

What will this return?

```
break isSpace "hello world"
```

Questions?

Knowledge check

1. Implement `span` yourself. (Challenging: consider a let-in along with if-expression)
2. Implement `break` in terms of `span`.
3. Give an example of a predicate and function in which the first return value of `span` will be the same as the result of `filter`.

KC 5 answer 1

```
span' :: (a -> Bool) -> [a] -> ([a], [a])
span' _ [] = ([], [])
span' f (x : xs) =
    if f x then
        let (t, rem) = span' f xs
        in (x : t, rem)
    else
        ([], x : xs)
```

Here, we check to see if `f x` is true. If it's not, we just return all the remaining values without processing them.

If it is, we recursively run `span'` on the rest of the values, but we also add the value `x` that `f` was true on.

Further practice: implement it using guards instead of an if expression.

KC 5 answers 2 and 3

2. `break' f = span (not . f)`

Here, we're saying "calling `break'` on some function `f` is the same as calling `span'` on `not` composed with that function. `break` just keeps running as long as `f` is false, rather than true.

3. `fst $ span (<10) [1,2,3,10,11,12]` will return the same result as `filter (<10) [1,2,3,10,11,12]`. However, if the list was `[1,2,3,10,11,12,1,2,3]` the results would not be the same.

Questions?

fun lisp history fact:

There are computer systems which were contemporary with early unix-based systems that used Lisp as the main shell language, and which had special hardware for running Lisp programs.

They were called [Lisp machines](#)

Imagine if you used lisp to write shell commands instead of bash or cmd. So like `(run 'some-program' arg1 arg2) .`

Personally, I think Lisp would be a great shell language, so I find it dissappointing that these went away.